Assignment 1

1. What is RMI?

Answer:
RMI (Remote Method Invocation) is a Java API that enables communication between Java objects located in different JVMs, allowing methods to be invoked on remote objects as if they were local.

🔁 How RMI Works in the Example

We have four components:

1. **Remote Interface (Hello)**
2. **Remote Object Implementation (HelloImpl)**
3. **Server (RMIServer)**
4. **Client (RMIClient)**

🔄 Execution Flow

✅ Step 1: Define Remote Interface

```java
public interface Hello extends Remote {
    String sayHello() throws RemoteException;
}
```

- This interface extends java.rmi.Remote, marking it as remote.
- The method sayHello() is declared to throw RemoteException, which is required for RMI communication.

✅ Step 2: Implement Remote Object

```java
public class HelloImpl extends UnicastRemoteObject implements Hello {
    public String sayHello() throws RemoteException {
        return "Hello from the RMI server!";
    }
}
```

- The class implements the remote interface Hello.
- It extends UnicastRemoteObject, which allows the object to receive incoming RMI calls.

- The method sayHello() contains the logic that will be executed remotely by the client.

✅ Step 3: Start the RMI Server

```java
Registry registry = LocateRegistry.createRegistry(1099);
registry.rebind("HelloService", obj);
```

- The server:
  - Creates a local RMI registry on port 1099.
  - Registers the remote object (obj) under the name "HelloService".
- Now, the object is ready to be found by clients and invoked remotely.

✅ Step 4: RMI Client Calls Remote Method

```java
Hello stub = (Hello) registry.lookup("HelloService");
String response = stub.sayHello();
```

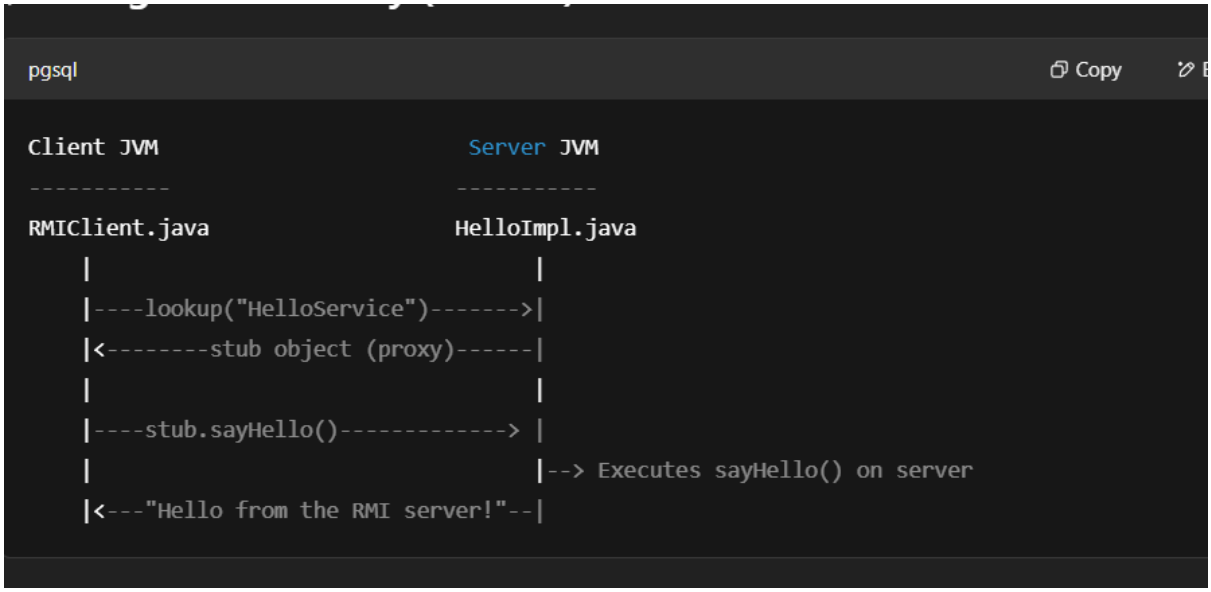
- The client:
  - Contacts the RMI registry on the server machine (localhost in this case).
  - Looks up "HelloService" and receives a **stub** (proxy object).
  - Calls sayHello() on the stub.
  - RMI automatically handles the network communication — sending the method call over the network, executing it on the server, and returning the result.

✅ Step 5: Communication Behind the Scenes

Here's what actually happens under the hood:

1. **Client uses stub**: Client holds a stub (auto-generated proxy) for the remote object.
2. **Method call sent over network**: The stub sends the request to the RMI server over TCP/IP.
3. **RMI server handles it in a thread**: A thread on the server accepts the call, passes it to the actual implementation (HelloImpl).
4. **Result returned to client**: The result ("Hello from the RMI server!") is sent back to the client.

📌 **Diagram Summary (Textual)**



2. What is the method that is used by the RMI client to connect to remote RMI servers?

Answer:
The method used is Naming.lookup(String url), which returns a reference to a remote object.

---

3. How does the distributed garbage collector manage disconnections on the client side?

Answer:
Java RMI uses **leases** to track client references to remote objects. When a client disconnects, its lease expires, and the distributed garbage collector eventually frees the remote object if no other clients hold references.

---

4. What is the relationship between RMI and CORBA?

Answer:
Both RMI and CORBA are used for distributed computing. RMI is Java-specific and easier to use in Java-based applications, while CORBA is language-independent and supports communication across different languages like C++ and Python.

---

5. What is a Remote Object?

Answer:
A remote object is an object whose methods can be invoked from another JVM. It must implement the Remote interface and be exported to receive incoming calls.

---

6. What is a Server Object?

Answer:
The server object is the implementation of the remote interface that is registered with the RMI registry. It provides the actual logic that the client accesses remotely.

---

7. What is rmiregistry?

Answer:
rmiregistry is a tool provided by Java to register remote objects so that clients can look them up using names. It acts as a naming service.

---

8. What are the different types of classes used in RMI?

Answer:

- **Remote Interface**
- **Remote Object (Implementation)**
- **Stub (client-side proxy)**
- **Skeleton (server-side dispatcher, older RMI)**
- **Client class**
- **Server class**

---

9. What is the main purpose of Distributed Object Applications in RMI?

Answer:
To enable method calls on objects in different JVMs/machines, allowing distributed systems to work together as if all components were local.

---

10. How does the communication with remote objects occur in RMI?

Answer:
The client calls methods on a **stub** (proxy), which communicates over the network with the **skeleton/server**, which invokes the actual method and sends the result back.

---

11. What are the steps involved in RMI distributed applications?

Answer:

1. Define a remote interface
2. Implement the interface on the server
3. Compile and generate stubs (older versions)
4. Start the RMI registry

5. Bind the object in the registry
6. Client looks up and invokes methods

---

12. What is the use of java.rmi.Remote Interface in RMI?

Answer:
It marks an interface as being eligible for remote method calls. All remote interfaces must extend java.rmi.Remote.

---

13. Write a program to show the remote interface using RMI.

java

CopyEdit

```
// File: MyRemote.java

import java.rmi.*;


public interface MyRemote extends Remote {

   String sayHello() throws RemoteException;

}
```

This defines a simple remote interface with one method sayHello() that can be invoked remotely.

---

14. Why are stubs used in RMI?

Answer:
A stub is a client-side proxy that represents the remote object. It forwards method calls from the client to the server over the network.

---

15. What is the function or role of skeleton in RMI?

Answer:
In pre-Java 2 versions, the skeleton was a server-side helper that received method calls from the stub and dispatched them to the actual implementation. Modern RMI doesn't require skeletons explicitly.

---

16. How does dynamic class loading happen in RMI?

Answer:
RMI can dynamically load classes from a specified codebase URL at runtime if a required class is not

available locally, using Java's class loading mechanism. This is enabled by setting the java.rmi.server.codebase property.

Assignment 2

✅ 1. What is CORBA?

Answer:
CORBA (Common Object Request Broker Architecture) is a standard defined by the OMG (Object Management Group) that enables software components written in multiple languages and running on different machines to work together.

✅ 2. What does CORBA provide?

Answer:
CORBA provides:

- Language-independent communication between objects
- An Object Request Broker (ORB) to handle method invocation
- Interface Definition Language (IDL) for defining object interfaces
- Interoperability between different platforms and languages

✅ 3. What does Java offer CORBA programmers?

Answer:
Java provides a built-in ORB and tools (idlj) for generating Java bindings from IDL files. It also includes libraries like org.omg.CORBA to develop CORBA-compliant applications.

✅ 4. Which protocol is used for invoking methods on CORBA objects over the internet?

Answer:
IIOP (Internet Inter-ORB Protocol) is used to allow CORBA objects to communicate over the internet.

✅ 5. Name some of the CORBA development tools.

Answer:

- idlj (IDL-to-Java compiler)
- VisiBroker
- JacORB

- TAO (The ACE ORB)
- OmniORB

✅ 6. Explain Naming Service in CORBA.

Answer:
The CORBA Naming Service is used to bind and locate distributed objects by name, similar to RMI's rmiregistry.

✅ 7. What are ORBlets?

Answer:
ORBlets are lightweight Object Request Brokers used in embedded or resource-constrained environments, allowing CORBA objects to run on small devices.

✅ 8. What is an Object Implementation in ORB Architecture?

Answer:
It is the actual server-side class that implements the methods defined in the IDL interface.

✅ 9. What is OMG IDL?

Answer:
OMG IDL (Interface Definition Language) is a language-neutral syntax used to define interfaces for CORBA objects, which can then be mapped to specific languages like Java or C++.

✅ 10. What is the Call Back Mechanism?

Answer:
It's a technique where a client registers a callback object on the server, allowing the server to invoke methods on the client asynchronously.

✅ 11. What is the Event Service in CORBA?

Answer:
CORBA Event Service provides a publish-subscribe mechanism where consumers receive events generated by suppliers (publishers) either synchronously (pull) or asynchronously (push).

✅ 12. What is the Callback Mechanism in CORBA?

Answer:
Same as Q10 – it allows the server to make method calls on client-registered objects, enabling asynchronous communication.

✅ 13. What is the main difference between RMI and CORBA?

Answer:

Feature	RMI	CORBA
Language	Java-only	Multi-language (C++, Python)
Protocol	JRMP	IIOP
Interface	Java Interface	IDL
Integration	Java-specific	Heterogeneous systems

✅ 14. Draw the Class Hierarchy in CORBA Application

pgsql

CopyEdit

```
        org.omg.CORBA.Object
                |
    YourIDLInterface (Generated from IDL)
                |
  YourIDLInterfacePOA (Portable Object Adapter class)
                |
      YourServerImplementation
```

✅ 15. What does CORBA offer Java Programmers?

Answer:

- Interoperability with non-Java components
- IDL to Java mapping
- Java ORB and Naming Service
- Portable object adapters (POA)

✅ 16. What types of event channel models does the Event Service provide?

Answer:

- **Push Model:** Supplier sends events to consumers
- **Pull Model:** Consumer pulls events from the supplier

✅ 17. Give the diagrammatic representation of Pull and Push models.

Push Model:

rust

CopyEdit

Supplier -----> Event Channel -----> Consumer

Pull Model:

lua

CopyEdit

Consumer <----- Event Channel <----- Supplier

✅ 18. What is URL Naming Service?

Answer:
URL Naming Service maps CORBA object references to URLs, allowing them to be resolved using internet-based naming mechanisms.

✅ 19. Explain the Bootstrapping technique in RMI.

Answer:
Bootstrapping in RMI refers to how a client initially obtains a reference to a remote object, usually via the RMI registry using Naming.lookup("rmi://...").

✅ 20. What are the CORBA services?

Answer:
CORBA Services include:

- Naming Service
- Event Service
- Transaction Service
- Notification Service
- Security Service

- Lifecycle Service

✅ **21. What are all the requirements needed to build a CORBA program?**

Answer:

- IDL file to define interfaces
- IDL compiler (e.g., idlj)
- ORB (e.g., Java ORB, TAO)
- Naming Service
- Language binding (e.g., Java classes from IDL)
- Stub and skeleton classes
- Client and Server code

Assignment 3

1. What is MPI?

MPI (Message Passing Interface) is a standardized, portable message-passing system designed for parallel computing across **distributed memory systems** (e.g., clusters). It allows processes on different nodes to communicate using messages.

---

2. What is OpenMP?

OpenMP (Open Multi-Processing) is an API that supports **multi-threaded** programming in shared memory environments (like multicore CPUs). It uses **compiler directives** to parallelize code, primarily in C, C++, and Fortran.

---

3. What is the difference between OpenMP and OpenMPI?

Feature	OpenMP	OpenMPI
Model	Multithreading	Multiprocessing
Memory	Shared memory	Distributed memory
Use case	On a single machine (multi-core)	Across multiple machines or cores
Implementation	Compiler-based (pragma directives)	Library-based (explicit message passing)

---

4. How can OpenMP threads permanently allocate memory on GPUs?

OpenMP 4.5+ supports offloading to GPUs using target directives. To allocate memory persistently:

- Use target enter data and target exit data to manage device memory explicitly.

---

5. Is OpenMP multiprocessing or multithreading?

OpenMP is **multithreading**. It creates threads within a single process to execute code in parallel.

---

6. Is OpenMP parallel or concurrent?

OpenMP is **parallel**. It runs threads simultaneously on multiple cores.

---

7. How many threads can OpenMP use?

The number depends on:

- Number of CPU cores
- The environment variable OMP_NUM_THREADS
- What you set in code (omp_set_num_threads(n))

---

8. What are the different types of synchronization in OpenMP?

- #pragma omp barrier
- #pragma omp critical
- #pragma omp atomic
- #pragma omp ordered
- Locks (omp_set_lock, omp_unset_lock)

---

9. Does OpenMP use multiple cores?

Yes, it is designed to use **multiple CPU cores** in shared memory systems.

---

10. What is the advantage of OpenMP?

- Simple to use with pragma directives
- No need to manage threads manually
- High performance on shared memory systems
- Scales well on multicore CPUs

---

11. How many cores does OpenMP use?

It uses as many cores as allowed by:

- System availability
- OMP_NUM_THREADS or omp_set_num_threads()

---

12. What is default in OpenMP?

- The default number of threads is usually equal to the number of **available logical cores**.
- Default data sharing is **shared** unless specified.

---

13. What is the limit of threads?

- System dependent (typically up to the number of cores or logical processors)
- Soft limit via OMP_NUM_THREADS
- Hard limit via OS or hardware constraints

---

14. How many threads can be active?

As many as the system supports concurrently, but typically limited by the number of CPU cores or logical processors.

---

15. Can OpenMP be used on more than one node?

No. OpenMP works only on **shared memory systems**, so it's limited to a single node.

---

16. Is OpenMP a compiler?

No. It's an **API** with **compiler support**, not a compiler itself.

---

17. Which compiler supports OpenMP?

- GCC (gcc, g++, gfortran)
- Clang (partial support)
- Intel compilers (icc, icpc)
- Microsoft Visual C++

---

18. Is OpenMP shared memory or distributed memory?

OpenMP is **shared memory**.

---

19. How many processors does OpenMP have?

It uses as many as the system provides — determined at runtime.

---

20. What is thread in OpenMP?

A thread is a unit of execution created by OpenMP to run parts of your program in parallel.

---

21. What is master thread in OpenMP?

The **master thread** is the original thread that spawns all other threads in a parallel region and often controls overall program flow.

---

22. What are the disadvantages of OpenMP?

- Limited to shared memory systems (not scalable to clusters)
- Debugging parallel bugs can be difficult
- Non-portable across systems without OpenMP support
- Potential for race conditions

---

23. What is the difference between thread and process in OpenMP?

Thread	Process
Shares memory	Has separate memory
Lightweight	Heavier, more overhead
Used in OpenMP	Used in MPI

---

24. What are the primary components of OpenMP?

- **Compiler directives** (e.g., #pragma omp parallel)
- **Runtime library routines** (e.g., omp_get_thread_num())
- **Environment variables** (e.g., OMP_NUM_THREADS)

Assignment 4

1. What is Berkeley Algorithm?

The **Berkeley Algorithm** is a clock synchronization technique used in distributed systems where no machine has an authoritative time source. A master node polls all other nodes, averages their time (after adjusting for propagation delay), and sends correction values to synchronize clocks.

---

2. What is the importance of Berkeley Algorithm?

It ensures that all nodes in a distributed system operate on a **synchronized time**, which is crucial for coordinated tasks, logging, and event ordering when there's no access to an external time server.

---

3. How is Berkeley Algorithm useful?

- Provides **synchronized time** without relying on an external clock.
- Tolerant of **clock drift** and minor faults.
- Helps in maintaining **consistency** and **order of events** across systems.

---

4. How is clock synchronization achieved through Berkeley Algorithm?

1. A **master node** sends time requests to all slaves.
2. Slaves respond with their local time.
3. Master calculates round-trip delays and the **average time** (excluding outliers).
4. Master sends **clock adjustments** (not absolute time) to slaves.
5. All nodes (including master) adjust their clocks accordingly.

---

5. Which algorithm is used for clock synchronization?

Common algorithms:

- **Cristian's Algorithm**
- **Berkeley Algorithm**
- **NTP (Network Time Protocol)**

---

6. How does Berkeley algorithm achieve fault-tolerant average and better synchronization?

- It **discards extreme values** (outliers) from faulty or delayed nodes before averaging.
- Adjusts for **network delay**.
- Uses **corrections** (deltas) instead of absolute time to reduce clock jumps.

---

7. What is the difference between Berkeley and Cristian's algorithm?

Feature	Berkeley Algorithm	Cristian's Algorithm
Time Source	Internal (master polls)	External time server
Updates	Average of all nodes	Server provides time
Fault tolerance	High (excludes outliers)	Depends on server accuracy
Time setting	Relative (correction)	Absolute time from server

---

8. What is the function of clock synchronization?

To ensure **consistent time** across all nodes in a system so that:

- Distributed processes coordinate effectively.
- Logs/events are accurately ordered.
- Data consistency and transaction integrity are maintained.

---

9. What is the accuracy of clock synchronization?

- Depends on algorithm, network latency, and clock drift.
- Berkeley can typically achieve accuracy in **milliseconds**, but not as precise as **NTP**.

---

10. What are the two methods used for time synchronization?

1. **External Synchronization**: Syncing with an authoritative time server (e.g., NTP, Cristian's).
2. **Internal Synchronization**: Nodes sync among themselves (e.g., Berkeley Algorithm).

---

11. What is an example of clock synchronization?

In distributed databases or file systems like Hadoop, nodes use time synchronization to **timestamp operations** consistently.

---

12. What is Berkeley Algorithm used for?

Used for **internal clock synchronization** in a distributed system where no reliable external clock exists.

---

13. Is Berkeley algorithm active or passive?

Active, as the master **initiates communication**, collects times, and sends back corrections.

---

14. What is the formula for propagation time in Berkeley Algorithm?

Propagation time ≈

(T_reply received - T_request sent)−processing delay2\frac{\text{(T_reply received - T_request sent)} - \text{processing delay}}{2}2(T_reply received - T_request sent)−processing delay

This estimates one-way delay for time adjustment calculations.

---

15. What are the benefits of clock synchronization?

- Accurate ordering of events
- Consistent data timestamps
- Reliable scheduling and coordination
- Reduced errors in distributed applications

Assignment 5

1. What is Mutual Exclusion?

Mutual Exclusion ensures that **only one process** at a time can access a **critical section** (shared resource) in a distributed or concurrent system to avoid data inconsistency and race conditions.

---

2. What are the different mutual exclusion algorithms available in distributed systems?

Some widely used **distributed mutual exclusion algorithms** include:

- **Ricart-Agrawala Algorithm**
- **Lamport's Algorithm**
- **Maekawa's Algorithm**
- **Suzuki-Kasami's Token-Based Algorithm**
- **Raymond's Tree-Based Algorithm**
- **Singhal's Heuristic Algorithm**

---

3. What are the requirements of mutual exclusion?

1. **Mutual Exclusion** – Only one process in the critical section at a time.
2. **Progress** – If no process is in the CS, one requesting process must be allowed to enter.
3. **Bounded Waiting** – There is a limit on how long a process waits before entering CS.
4. **Fairness** – No starvation; requests should be served in order.
5. **Fault Tolerance** – Should handle message loss, process failure, etc. (in distributed systems).

---

4. Classify Distributed Mutual Exclusion Algorithms:

a. Non-Token Based Algorithms:

- Lamport's Algorithm
- Ricart-Agrawala Algorithm
- Maekawa's Algorithm

b. Token-Based Algorithms:

- Suzuki-Kasami Broadcast Algorithm
- Raymond's Tree-Based Algorithm

---

5. Differentiate between Token-Based and Non-Token-Based Algorithms:

Feature	**Token-Based**	**Non-Token-Based**
Access Method	Token (unique privilege message)	Voting or timestamp-based
Message Overhead (avg)	Lower	Higher
Failure Sensitivity	Token loss is problematic	Handles failure better
Example Algorithms	Suzuki-Kasami, Raymond's	Lamport's, Ricart-Agrawala

---

6. What are different Non-Token-Based Algorithms in Distributed Systems?

- **Lamport's Logical Clock Algorithm**
- **Ricart-Agrawala Algorithm**
- **Maekawa's Algorithm**
- **Singhal's Heuristic Algorithm**

---

7. What are different Token-Based Algorithms in Distributed Systems?

- **Suzuki-Kasami's Broadcast Algorithm**
- **Raymond's Tree-Based Algorithm**
- **Token Ring Algorithm**
- **Singhal's Tree-Based Algorithm**

---

8. What are the different performance measures of mutual exclusion algorithms?

- **Message complexity** (messages per CS entry)
- **Synchronization delay** (time to enter CS)
- **Throughput** (CS accesses per time unit)
- **Fairness** (order of access)
- **Fault tolerance** (resilience to failure)
- **Recovery overhead**

---

9. Comparative Performance Analysis of Mutual Exclusion Algorithms:

Algorithm	Message Complexity	Delay	Fault Tolerance	Fairness	Type
Lamport's	3(n-1)	High	Yes	Yes	Non-Token
Ricart-Agrawala	2(n-1)	High	Yes	Yes	Non-Token
Maekawa's	O(√n)	Medium	Yes	Moderate	Non-Token
Suzuki-Kasami	1 (broadcast)	Low	No (if token lost)	Yes	Token-Based
Raymond's	log(n)	Low	Moderate	Yes	Token-Based

Assignment 6

1. What is Election Algorithm?

An **election algorithm** is a technique used in **distributed systems** to **elect a coordinator or leader** among distributed processes. The leader manages centralized tasks like mutual exclusion, resource allocation, etc.

---

2. Name different election algorithms:

- **Bully Algorithm**
- **Ring Algorithm**
- **Chang and Roberts Algorithm**
- **LeLann–Chang–Roberts (LCR) Algorithm**
- **Hirschberg-Sinclair Algorithm**
- **Randomized Leader Election**
- **FloodMax Algorithm**

---

3. What is Bully and Ring Algorithm?

- **Bully Algorithm**:
    - Initiated by a lower-ID process.
    - Higher-ID processes respond and take over the election.
    - The highest-ID process becomes the coordinator.
    - It "bullies" others by overriding lower-ID initiators.
- **Ring Algorithm**:
    - Processes are organized in a logical ring.
    - Election messages circulate until the highest-ID is determined.
    - That process is elected leader and sends a coordinator message.

---

4. What does an election algorithm do?

It **selects a leader (coordinator)** among multiple nodes or processes in a distributed system, especially after **leader failure** or system start-up.

---

5. Why are election algorithms normally needed in a distributed system?

- To **replace failed coordinators**
- For **resource management**
- To ensure **consistency** and **coordination**
- To **avoid conflicts** in critical section handling

---

6. What is leader election algorithm and why do we need it?

A **leader election algorithm** elects one node as the **central coordinator** among peers. We need it for:

- Task delegation
- Centralized decision making
- Synchronization and control

---

7. How many types of messages are there in election algorithm?

Typically 2 to 4 types, depending on the algorithm. Common ones include:

- **Election Message**
- **OK or Reply Message**

- **Coordinator Announcement**
- **Alive or Heartbeat (optional in fault-tolerant versions)**

---

8. What are the features required for election algorithms?

- **Uniqueness**: Only one leader elected
- **Fairness**: All nodes have equal chance
- **Fault Tolerance**: Should work even if some nodes fail
- **Efficiency**: Minimal time and message overhead
- **Scalability**: Should perform well with large systems

---

9. What is the purpose of election system?

To **dynamically choose a leader** in a distributed environment for:

- Coordination
- Failure recovery
- Ensuring no multiple leaders exist (to avoid conflicts)

---

10. What is the time complexity of leader election?

- **Bully Algorithm**: Worst case **$O(n^2)$** (n = number of processes)
- **Ring Algorithm**: **O(n)**
- **Chang and Roberts (Ring Variant)**: **O(n log n)** (average case with optimization)
- **FloodMax**: **O(log n)** rounds with **O(n log n)** messages

Assignment 7

1. What is Web Service?

A **Web Service** is a **software system** designed to **support interoperable machine-to-machine interaction** over a network using standard protocols (like HTTP). It allows different applications to **communicate and exchange data**, regardless of platform or language.

---

2. What are different types of Web Services?

1. **SOAP (Simple Object Access Protocol)** Web Services
2. **REST (Representational State Transfer)** Web Services
3. **XML-RPC** (Remote Procedure Call using XML)

4. **JSON-RPC** (Remote Procedure Call using JSON)

---

3. What is SOAP?

SOAP (Simple Object Access Protocol) is a **protocol** used for exchanging structured information in web services using **XML**. It works over HTTP, SMTP, etc., and supports strict **standards** and **security**.

---

4. What is REST?

REST (Representational State Transfer) is an **architectural style** that uses **standard HTTP methods** (GET, POST, PUT, DELETE) for communication. RESTful web services are lightweight, stateless, and often use **JSON** or **XML** for data exchange.

---

5. Explain Web Services Architecture:

Web services architecture is typically a **three-component model**:

- **Service Provider**: Offers the service.
- **Service Requestor**: Consumes the service.
- **Service Registry**: A searchable directory where services can be published and discovered.

The architecture supports operations like:

- **Publishing** (provider → registry)
- **Finding** (requestor → registry)
- **Binding/Invoking** (requestor ↔ provider)

---

6. Explain the concept of Service Provider:

The **Service Provider** hosts the **web service**, describes it using **WSDL (in SOAP)** or **OpenAPI (in REST)**, and responds to incoming requests.

---

7. Explain the concept of Service Requestor:

The **Service Requestor** (or **client**) is the application or system that **searches, locates**, and **invokes** the web service.

---

8. Explain the concept of Service Registry:

The **Service Registry** is a **central directory** (like UDDI – Universal Description, Discovery, and Integration) that allows providers to publish their services and requestors to discover them.

---

9. Differentiate SOAP and REST:

Feature	SOAP	REST
Protocol	Protocol	Architectural style
Data Format	XML only	XML, JSON, HTML, etc.
Transport	HTTP, SMTP, TCP, etc.	HTTP only
Complexity	Heavyweight	Lightweight
Performance	Slower	Faster
Security	Built-in (WS-Security)	Relies on HTTPS
State	Supports stateful operations	Stateless
Standard Compliance	Strict (WSDL, UDDI, SOAP schema)	Flexible